Data Science in the age of AI (Part 1): The machine wrote my code (and that’s not the problem)

Why your value isn’t in the code or building ML models. And it actually never was.

JOSE PARREÑO GARCIA
OCT 04, 2025

7 2 Share

This week, I asked one of my reports how they built a prototype *so fast*.

We had just had a casual chat the day before, tossing around the idea of using machine learning to explore feature importance. Nothing urgent or production ready. Just a “*Could we get a clearer sense of which features might actually matter, without spending days manually poking around?*”

The next day, they had done pretty much everything. It is true that the data was ready, but the model was built, they had done all the visuals, applied SHAP… in other words, an actual proper deep dive.

So I asked: *"Wait... how much of this did you actually write?"*
They smiled. *"Honestly? Not much. ChatGPT did most of it."*

And the truth is, that was totally fine. The prototype worked, unblocked discussion and saved hours. I honestly did not care how they had done the work. It was a simple but effective piece of work, and that's what I wanted.

However, it also triggered something deeper in me.

> **If the machine can write the code... what's left for us to do?**
>
> **If LLMs can do the first 80% of the work — fast, and well enough — then what actually sets a data scientist apart today?**

In this post, I will cover those questions and explore how the role of data scientists is evolving, not because of AI, but through it.

Senior Data Science Lead is a reader-supported publication. To receive new posts and support my work, consider becoming a free or paid subscriber.

Blog series

- 📌 Data Science in the age of AI (Part 1): The machine wrote my code (and that's not the problem) - ***(this post!)***
- Data Science in the age of AI (Part 2): If everyone can build a model, what are Data Scientists for? - *(coming soon!)*
- Data Science in the age of AI (part 3): Prompting is not building AI systems - *(coming soon!)*
- Data Science in the age of AI (part 4): The science we can't afford to lose - *(coming soon!)*
- Data Science in the age of AI (part 5): The future hybrid - *(coming soon!)*

What this blog will cover

- **The automation trend *didn't* start with LLMs.** Why code-generation is just the latest chapter in a longer story of abstraction and tooling in data science.
- **LLMs: The good, the bad, and the mostly boilerplate.** Recapping that LLMs are not dark magic.
- **Why this doesn't make data scientists obsolete.** What the machine can't do (yet): judgment, prioritisation, critical thinking.
- **The shift in what makes a data scientist valuable.** If code is cheap, context and clarity is everything.

- **A word of caution for junior data scientists.** Why LLMs make it easier to look smart, but harder to *become* smart.
- **Closing thoughts.** This shift isn't a threat. It's an opportunity to focus on the parts of the job that actually matter.

Let's start!

The automation trend *didn't* start with LLMs.

Long before ChatGPT or Claude Sonnet started generating code, data scientists were automating the boring stuff. We had wrappers in `sklearn`, pipelines in PySpark, hyperparameter tuning with Optuna, AutoML platforms like H2O, and simpler ways of running MLOps through tools like Databricks.

I've lived through this shift.

Before all that, I had to write `sklearn.pipeline()` from scratch. Hyperparameter tuning meant quite a few `for` loops. I could only afford to train a couple of models, not run a full-blown AutoML sweep. And model deployment meant manually exporting to ONNX (or something equally fidly), then wrapping it all up in a container.

I know how these tools made my life so much easier. Thanks to these, we templated feature engineering, abstracted training loops, reused ETL scaffolds, and ran

experiments through MLflow or Airflow DAGs. The work was becoming faster, safer, and more repeatable because someone wrote the boilerplate so we didn't have to.

The goal was always the same:

> Spend less time writing glue code.
>
> Spend more time thinking.

Then came LLMs, and they made that process *instant*.

LLMs didn't invent automation.

They just *democratised* it — and made it conversational.

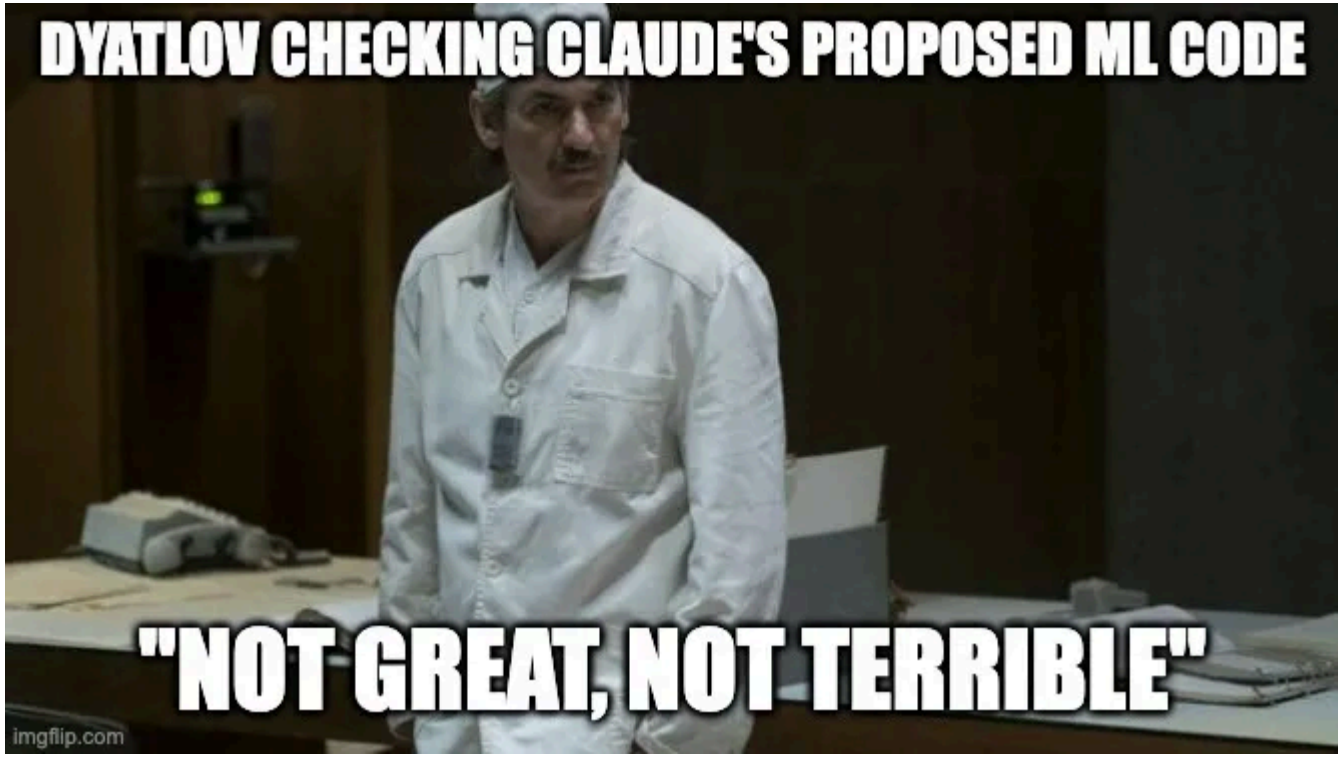
For example:

- *"Write a PySpark ETL for deduplicating daily hotel feeds."*
- *"Build me a LightGBM classifier with early stopping and log loss."*
- *"Turn this spaghetti into a modular sklearn pipeline with docstrings and tests."*

Done. In seconds. With examples.

This isn't new. It's definitely not magic, it's more an autocomplete on steroids.

And it works because the LLMs were trained on exactly the kind of boilerplate we were already trying to automate. If we had already wrapped repetitive ML work in templates and tools, LLMs are simply the next wrapper.



LLMs: The good, the bad, and the mostly boilerplate code

The good

If you have used one seriously in your workflow, you already know that they can confidently handle the following:

- **Boilerplate code**: pipelines, encoders, configs, evaluation loops
- **Starter notebooks**: for EDA, model prototyping, even light dashboards
- **Code translation**: SQL to pandas, pandas to PySpark, old scripts to new APIs
- **Helper code**: unit tests, docstrings, visualisations, basic error handling
- **Debugging support**: “what does this warning mean?”

For many data scientists, LLMs are now part of the “furniture”. You know what you want. You describe it. You get a draft that is often 80% of the way there. Then you clean it up, adapt it, and move on.

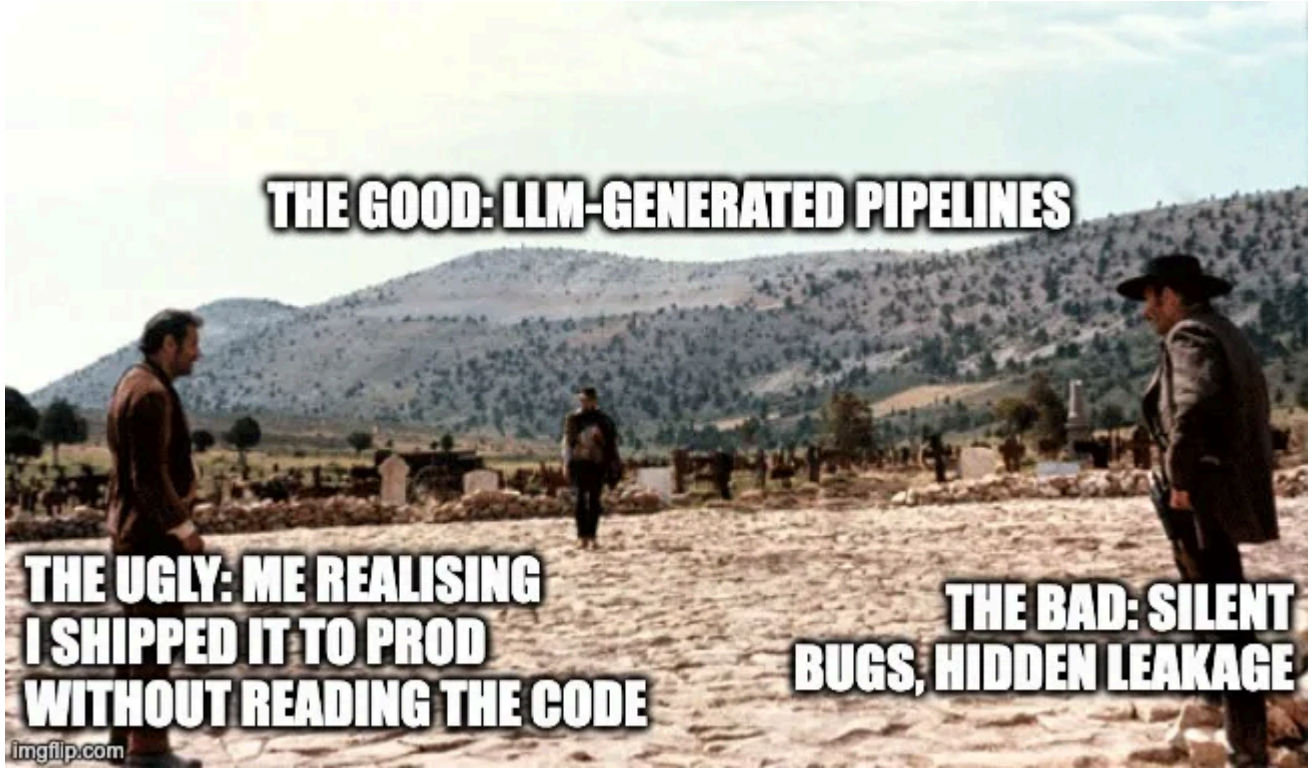
That last bit matters. As a Senior Data Scientist, I always read the code being spit out by the LLM. I don’t just copy-paste-run. LLMs for a boilerplate first draft yes… but for something more complex… maybe not yet.

However, if the machine can write that much of the first draft (and might maybe write so much better code in the future) — **what’s left for you to do?**

The bad

For all the hype, LLMs still stumble on the stuff that actually makes a data scientist valuable. They struggle with:

- **Ambiguity** — What is the *real* problem we are trying to solve?
- **Prioritisation** — Should we optimise for accuracy, fairness, speed, interpretability?
- **Contextual judgement** — That model looks great, but is it aligned with business constraints?
- **Domain nuance** — No, we don’t split revenue evenly between partners. That assumption breaks everything.
- **Ethics and trade-offs** — Bias, fairness, unintended consequences — all still very human responsibilities.



LLMs can make you faster. But they can't tell you what matters. Not yet (and maybe not ever).

That's where *you* come in.

Why this doesn't make data scientists obsolete

Writing code was never the job. Building models was never the reason you were paid good money.

At least, not the *whole* job. Not the part that matters most.

Sure, it's satisfying. Some of us enjoy building a good pipeline from scratch or tweaking that last hyperparameter (I am guilty of that, I love building ML models). But, the core value of a data scientist, especially in business settings, has never been about who can do deep learning or who can build a time series forecasting model.

The real value lies in understanding the *problem*.

In knowing *what* to build, *why* it matters, and *when* to stop.

Models are just the complex technical tool which can help you push for way better optimal solutions than a simple hand-crafted solution could. But, if you don't know what to build, why it matter and when it is good enough, then I can guarantee that a product person with a clear vision, will outperform your fancy ML model with some basic rules.

What makes you valuable

We all know LLMs can write code. They can even build decent machine learning models. But so could AutoML (open-source packages have been doing this for years).

What neither LLMs *nor* AutoML can do (among other things) is:

- **Choose the right business objective**
  It can optimise for click-through rate, but it won't ask whether revenue per user, long-term retention, or fairness across partners actually matters more.
- **Challenge a vague stakeholder request**
  When someone says, *"Just predict churn,"* the LLM won't push back and ask: *"Which kind of churn? Inactive users? Cancelled bookings? Or users we actually want to lose?"*
- **Ask the uncomfortable question no one else is asking**
  Like: *"Are we training this model on data that includes fraudulent bookings?"*
  Or: *"Does this experiment exclude 40% of users because of tracking gaps?"*
- **Choose the right trade-off between precision and latency**
  It can maximise F1-scores. But it won't know you need <200ms inference because this model powers real-time search ranking.
- **Tell you when the data is lying to you**
  That spike in revenue? A human remembers there was a partner integration bug that duplicated transactions. The LLM gleefully treats it as a signal.

These are the moments where real data science happens. When you come in and provide the judgment, the context, and the ability to ask better questions. You can

always return to the LLM and tweak what it provides based on your knowledge, but as you can see, you are past the boilerplate code.

You are into the space where value is created — not just executed.

This is the kind of work that engineers, analysts, and even stakeholders may struggle to do. And this is exactly where you, as a data scientist, still make a difference.

The shift in what makes a data scientist valuable.

We have ascertained that, if everyone can write code with an LLM, then code isn't a differentiator anymore. And if building a model is just a prompt away, then *building the model* isn't where your edge is, either.

So what is?

> Thinking clearly.
> Framing problems well.
> Designing smart experiments.
> Knowing the difference between signal and noise.

In my opinion, **the *value* is moving up the stack.**

What used to be a "nice to have" is now the job

Let's rewind five years.

You could make a solid career as a "backend-focused" data scientist who didn't love stakeholder meetings. Maybe you weren't great at framing problems, but you were the person who could build *the best model*. You knew your hyperparameters, you could wrangle Spark and, more importantly, you delivered something that no one else could.

That was enough.

Today? Well, you know what I have been saying... the value is not only in the modelling.

What used to be *bonus skills* now define whether your work lands or gets lost:

- **Clear problem framing**
  "What exactly are we trying to improve? Who defines success? Is this a prediction problem or a prioritisation one?"
- **Smart metric selection and segmentation**
  You don't just report an uplift. You know to break it down by country, device, acquisition channel and explain *why* one segment went backwards.
- **Translating business ambiguity into testable hypotheses**
  Stakeholder says: *"Make the homepage smarter."* You turn that into:
  "Would we like to predict whether a user prefers deals vs discovery and personalise accordingly?"
- **Stakeholder alignment and prioritisation**
  When resources are tight, you know how to define feasibility and potential from a Data Science point of view. This is the sort of thing that helps stakeholders align with your decisions and trust you.

- **Strategic storytelling with results**
  You don't just say *"model A beat model B by X% and we know that accuracy is better by Y%"*. You add value by connecting the dots: *"Model A wins because it aligns better with seasonal user intent especially for mobile users in low-cost markets."*

And yes, understanding the maths *still matters*

There is another shift worth calling out.

As boilerplate becomes free and fast, your actual technical understanding becomes more important, *not less*.

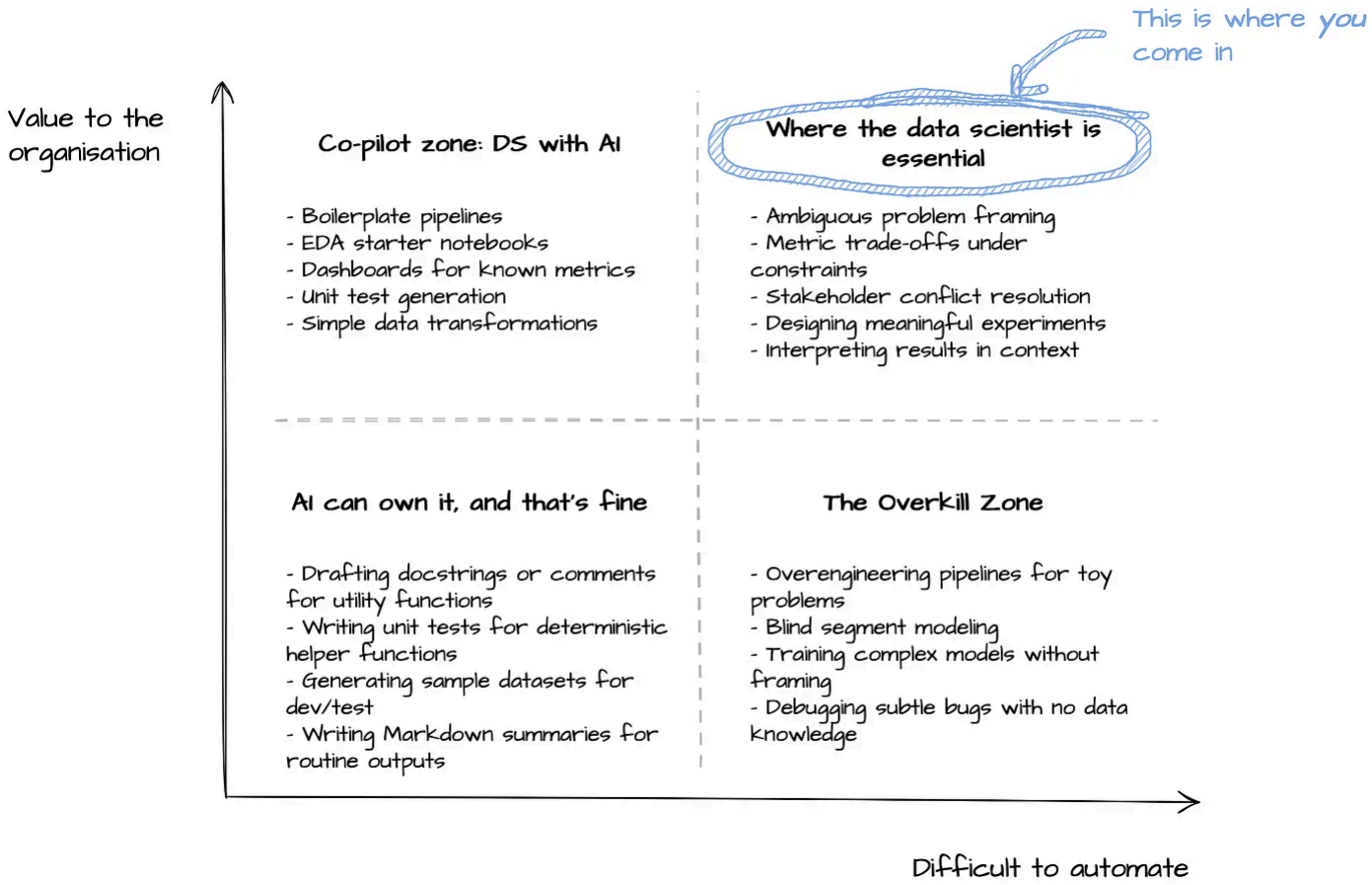
Because when you need to:

- Handle high-cardinality categorical features without exploding memory
- Decide when deep learning is overkill vs when XGBoost hits a ceiling
- Understand why a log-transform fixes skew in your residuals
- Spot data leakage that *looks* fine to an LLM
- Optimise inference latency without compromising precision

… you can't just prompt your way through it (or maybe you could, but without this knowledge, it will take a non Data Scientist an age).

Basically, LLMs can get anyone started. But, *experts still finish the job.*

This is where mathematical grounding, practical ML intuition, and scars from real experiments start to matter again. Because at the edge cases — where decisions get expensive — **real expertise still wins.**



A word of caution for junior data scientists

If you are early in your career, LLMs can feel like magic. They help you move faster, unblock problems, and build things you might not have known how to start from scratch. That is powerful. But it can also be misleading.

Just because something works doesn't mean you understand it. And if you rely too heavily on the LLM to generate your code, you might skip the hard and essential process of learning how to reason through problems yourself.

You don't become a strong data scientist by copying code that runs. You become one by understanding what that code is doing, and more importantly, *why* it's the right thing to do in that situation.

Be careful not to replace deep understanding with speed of delivery because the following could happen:

- Using the wrong evaluation metric for the business question
- Including target leakage without realising it
- Ignoring broken segment performance because overall results look good
- Misusing imputation or one-hot encoding in a way that quietly skews results
- Using incorrect transformations for certain distributions

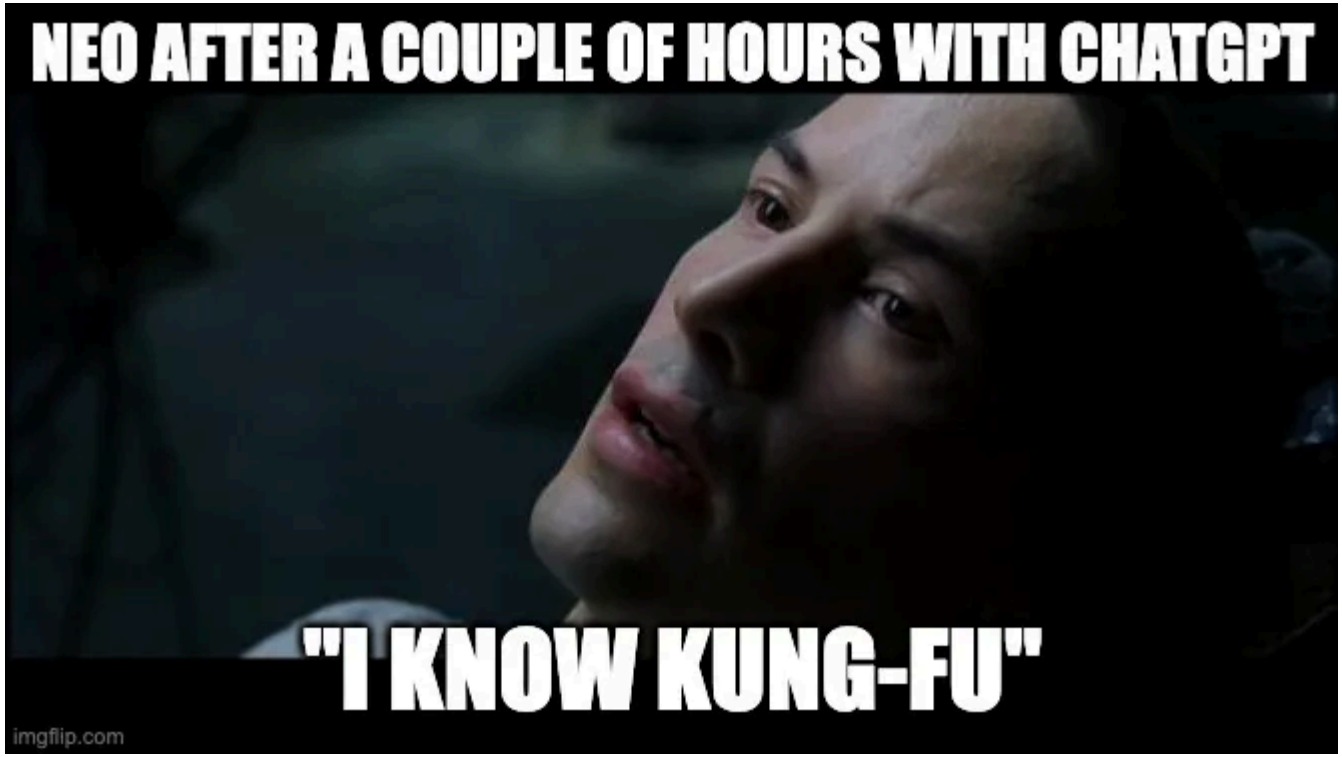
- Assuming that anything returned by an LLM is a best practice (it's often not)

The LLM makes it easy to get something that looks good enough to demo. **But looking competent and *being* competent are not the same thing.**

This doesn't mean you shouldn't use LLMs — you absolutely should. But treat them like a junior colleague with a fast brain and questionable judgment. Check their work. Ask questions. Read every line before you run it. Try to break it. And when something feels off, dig deeper.

Because the moment will come when someone asks you, *"Why did you choose this model?"* or *"Why is this segment underperforming?"* or *"Can we trust this result in production?"* And when that happens, you definitely cannot say *"Well, ChatGPT wrote it."*

At that point, *you* need to know.



Closing reflection

The job is changing. Not in the future. **Right now**.

LLMs have already reshaped what it means to be a data scientist. They have taken the scaffolding — the boilerplate, the translation, the setup — and made it instant. For me this is an amazing opportunity.

The reality is that, after the boilerplate is gone, the real work starts to happen: understanding problems, optimising models, designing experiments, navigating ambiguity, and translating numbers into decisions. If that's not the part of the job that excites you… now is a good time to reconsider.

But if it *is* the part that excites you — if you like wrestling with hard questions, figuring out what matters, and building things that actually change how a product works — then this shift is in your favour.

In a world where code is cheap and context is everything, **clarity, judgment, and domain understanding are your competitive advantage.**

> So use the machine. Let it make you faster. **But don't let it make you passive.**
>
> You are not here to write boilerplate.
>
> You are here to solve problems.

Now, I want to hear from you!

- Where do *you* think data scientists still add the most value?
- Have you caught an issue the LLM missed? Had a moment where good problem framing changed everything? Or maybe you've seen a team over-rely on generated

code and pay the price later?

- What's your take — are LLMs a shortcut to better work, or a crutch that's too easy to lean on?

Drop your stories, lessons, or counterpoints in the comments. I'd love to hear how *you're* navigating the shift 👇

And see you in the next post about *Data Science in the age of AI.* 👋

Further reading

If you are interested in more content, here is an article capturing all my written blogs!

All my articles, one convenient place

JOSE PARREÑO GARCIA • 15 OCTOBER 2024

Read full story →

7 Likes • 2 Restacks

Discussion about this post

Comments Restacks

Write a comment...

© 2026 Jose Parreño Garcia · Privacy · Terms · Collection notice
Substack is the home for great culture